

# **The Algorithmic Blackbox**

*Engineering a High-Frequency Trading Simulator  
and  
Autonomous Reinforcement Learning Agent*

**Mudil Goel**

Roll Number: 24B3932

*Department of Electrical Engineering  
Indian Institute of Technology Bombay*

February 3, 2026

**Wids Project: Algorithmic Trading & RL  
Submission: Week 1-3 Final Deliverable**

## Abstract

Modern financial markets are complex adaptive systems where price discovery emerges from the high-frequency interactions of algorithmic agents. This project, "The Algorithmic Blackbox," aimed to deconstruct this complexity by building a high-fidelity market simulator from first principles.

The project was executed in three phases. **Phase I (The Engine)** focused on engineering a Limit Order Book (LOB) matching engine capable of  $O(\log N)$  execution using dual-heap data structures. **Phase II (The Ecosystem)** involved populating this environment with heterogeneous agents—Noise Traders, Market Makers, and Momentum Traders—to reproduce stylized facts of financial time series such as fat tails and volatility clustering. **Phase III (The Brain)** integrated a Reinforcement Learning (RL) agent trained via Proximal Policy Optimization (PPO).

We demonstrate that while naive RL agents fall into local optima (the "safety trap"), agents trained with risk-aware reward shaping can successfully navigate high-volatility regimes, providing liquidity while managing inventory risk. This report documents the full architectural design, mathematical formulation, and experimental validation of the simulator.

## Contents

|          |                                   |          |
|----------|-----------------------------------|----------|
| <b>1</b> | <b>Project Overview</b>           | <b>1</b> |
| <b>2</b> | <b>Scope and Objectives</b>       | <b>1</b> |
| <b>3</b> | <b>Order Book Theory</b>          | <b>2</b> |
| <b>4</b> | <b>Data Structure Engineering</b> | <b>2</b> |
| 4.1      | Max-Heap for Bids . . . . .       | 2        |
| 4.2      | Min-Heap for Asks . . . . .       | 2        |
| <b>5</b> | <b>Matching Logic</b>             | <b>2</b> |
| <b>6</b> | <b>Agent-Based Modeling (ABM)</b> | <b>3</b> |

|           |                                                  |          |
|-----------|--------------------------------------------------|----------|
| <b>7</b>  | <b>The Agent Zoo</b>                             | <b>3</b> |
| 7.1       | Noise Traders . . . . .                          | 3        |
| 7.2       | Market Makers . . . . .                          | 3        |
| 7.3       | Momentum Traders . . . . .                       | 4        |
| <b>8</b>  | <b>Experimental Validation</b>                   | <b>4</b> |
| 8.1       | Stylized Fact 1: Fat Tails . . . . .             | 4        |
| 8.2       | Stylized Fact 2: Volatility Clustering . . . . . | 4        |
| <b>9</b>  | <b>MDP Formulation</b>                           | <b>4</b> |
| 9.1       | State Space ( $\mathcal{S}$ ) . . . . .          | 5        |
| 9.2       | Action Space ( $\mathcal{A}$ ) . . . . .         | 5        |
| <b>10</b> | <b>Reward Engineering</b>                        | <b>5</b> |
| 10.1      | The "Safety Trap" . . . . .                      | 5        |
| 10.2      | The Risk-Aware Solution . . . . .                | 5        |
| 10.3      | Final Policy Behavior . . . . .                  | 6        |
| <b>11</b> | <b>Summary of Achievements</b>                   | <b>6</b> |
| <b>12</b> | <b>Future Directions</b>                         | <b>6</b> |
| <b>A</b>  | <b>Core Engine (engine.py)</b>                   | <b>7</b> |
| <b>B</b>  | <b>Agents (agents.py)</b>                        | <b>7</b> |
| <b>C</b>  | <b>RL Environment (rl_environment.py)</b>        | <b>7</b> |

## List of Figures

## List of Tables

## 1 Project Overview

The digitization of financial markets has shifted trading from human-driven outcry pits to silicon-based Limit Order Books (LOBs). In this regime, "liquidity" is provided by algorithms and consumed by algorithms. To understand this ecosystem, one cannot simply analyze historical data; one must simulate the underlying dynamics.

This project builds an end-to-end quantitative research platform, comprising:

1. A high-performance **Matching Engine** that enforces market rules.
2. A **Simulation Core** that orchestrates time and events.
3. An **Agent Zoo** representing diverse market participants.
4. A **Deep Reinforcement Learning** framework for optimal execution.

## 2 Scope and Objectives

The project was divided into three weekly sprints, each with specific technical deliverables:

- **Week 1:** Implement the core matching logic (price-time priority) and data structures.
- **Week 2:** Model market microstructure. Generate realistic order flow using Brownian motion and feedback loops (Momentum). Validate against statistical benchmarks ("Stylized Facts").
- **Week 3:** Train an autonomous agent to profit in this synthetic market without incurring ruinous risk, using PPO (Proximal Policy Optimization).

Phase I: The Matching Engine

### 3 Order Book Theory

The Limit Order Book is the state of all outstanding buying and selling interest. It is defined by two lists:

- $B(t)$ : The list of buy orders (Bids), sorted descending by price.
- $A(t)$ : The list of sell orders (Asks), sorted ascending by price.

A trade occurs instantaneously when the inequality  $P_{bid}^{best} \geq P_{ask}^{best}$  is satisfied.

### 4 Data Structure Engineering

Naive implementations using Lists ( $O(N)$  insertion) are too slow for HFT simulation. We implemented a **Dual-Heap** architecture to ensure logarithmic time complexity.

#### 4.1 Max-Heap for Bids

Since the matching engine needs to constantly access the *highest* buy price, Bids are stored in a Max-Heap. Since Python's `heapq` is a min-heap, we store prices as negative values:

$$\text{Priority} = (-1 \times \text{Price}, \text{Timestamp})$$

#### 4.2 Min-Heap for Asks

Asks are stored in a standard Min-Heap to access the *lowest* sell price efficiently.

$$\text{Priority} = (\text{Price}, \text{Timestamp})$$

### 5 Matching Logic

The `match()` function is the core of the simulator. It implements **Price-Time Priority**:

1. **Price Priority:** Better prices match first.
2. **Time Priority:** At the same price, older orders match first.

Phase II: Agent Ecology

## 6 Agent-Based Modeling (ABM)

A market engine is useless without order flow. In Phase II, we populated the simulator with autonomous agents to generate realistic price dynamics.

## 7 The Agent Zoo

### 7.1 Noise Traders

Noise traders represent uninformed retail flow. They do not predict future prices but provide the baseline liquidity for the market.

- **Model:** Geometric Brownian Motion (GBM).
- **Logic:** They track a "Fair Value"  $S_t$  and place limit orders at  $S_t \pm \epsilon$ .
- **Role:** Preventing the order book from becoming empty.

### 7.2 Market Makers

Market Makers (MM) are liquidity providers who profit from the spread. We implemented a simplified \*\*Avellaneda-Stoikov\*\* strategy.

$$P_{quote} = P_{mid} \pm \frac{\delta}{2} - \gamma(I_t) \tag{1}$$

where  $I_t$  is the current inventory.

- If the MM is **Long** ( $I_t > 0$ ), they lower quotes to encourage selling.
- If the MM is **Short** ( $I_t < 0$ ), they raise quotes to encourage buying.

## 7.3 Momentum Traders

To generate realistic volatility, we introduced Trend Followers.

- **Logic:** Moving Average Crossover (SMA-10 vs SMA-50).
- **Feedback Loop:** If prices rise, they buy, pushing prices higher. This positive feedback loop creates "Fat Tails" (crashes and spikes).

## 8 Experimental Validation

We validated the realism of our simulator by checking for **Stylized Facts** of financial markets.

### 8.1 Stylized Fact 1: Fat Tails

Real market returns do not follow a Normal Distribution; extreme events are common.

- **Metric:** Kurtosis. (Normal Dist = 0.0).
- **Simulated Result:** Kurtosis = **12.45**.

This high kurtosis confirms our Momentum Traders successfully destabilized the market, creating realistic crash dynamics.

### 8.2 Stylized Fact 2: Volatility Clustering

We measured the autocorrelation of absolute returns. The simulation showed significant positive autocorrelation at Lag-1, indicating that high-volatility periods cluster together.

Phase III: Reinforcement Learning

## 9 MDP Formulation

We formulated the trading problem as a Markov Decision Process (MDP).

## 9.1 State Space ( $\mathcal{S}$ )

The agent observes a vector of 14 normalized variables:

- **LOB State:** Top 5 Bids and Top 5 Asks (relative to mid-price).
- **Internal State:** Current Inventory and Cash.
- **Market Context:** Current Spread and Volatility.

## 9.2 Action Space ( $\mathcal{A}$ )

We used a discrete action space to facilitate learning:

- **0 (HOLD):** Do nothing.
- **1 (BUY):** Market Buy 1 unit.
- **2 (SELL):** Market Sell 1 unit.

# 10 Reward Engineering

The most challenging aspect of Phase III was designing a reward function that prevents the agent from learning "lazy" strategies.

## 10.1 The "Safety Trap"

Initially, utilizing a simple PnL reward ( $R_t = \Delta PnL$ ), the agent converged to a policy of doing nothing to avoid transaction costs.

## 10.2 The Risk-Aware Solution

We engineered a composite reward function:

$$R_t = \underbrace{\Delta PnL}_{\text{Profit}} - \underbrace{\alpha |Inv_t|}_{\text{Inventory Risk}} - \underbrace{\beta (DD_t)}_{\text{Drawdown Risk}} \quad (2)$$

This forces the agent to keep inventory near zero (Mean Reversion) and avoid massive losses, rather than just maximizing raw profit.

### 10.3 Final Policy Behavior

In high-volatility regimes ( $\sigma = 5.0$ ), the agent learned a **Mean Reversion Strategy**:

- Buying when price deviations exceeded 2 standard deviations.
- Selling immediately upon profit to reduce inventory risk.
- **Win Rate:** 54% (vs 50% random baseline).

Conclusion and Future Work

## 11 Summary of Achievements

This project successfully delivered a full-stack algorithmic trading research platform.

1. **Robust Architecture:** The Dual-Heap engine proved stable under loads of 10,000+ orders.
2. **Realistic Data:** We successfully reproduced non-Gaussian market statistics without using external datasets.
3. **Autonomous Learning:** The RL agent demonstrated the ability to discover profitable strategies autonomously, provided the reward function is carefully shaped.

## 12 Future Directions

- **Execution Algorithms:** Implementing TWAP/VWAP execution agents.
- **Adversarial Training:** Training two RL agents to compete against each other for liquidity.
- **Latency Modeling:** Introducing network delays to simulate HFT infrastructure constraints.

## References

- [1] Avellaneda, M., & Stoikov, S. (2008). *High-frequency trading in a limit order book*. Quantitative Finance.
- [2] Schulman, J., et al. (2017). *Proximal Policy Optimization Algorithms*. OpenAI.
- [3] Towers, M., et al. (2023). *Gymnasium: Standard Interface for RL*.

## Code Appendix

### A Core Engine (engine.py)

```
1 class AdvancedOrderBook:
2     def __init__(self):
3         self.bids = [] # Max-Heap
4         self.asks = [] # Min-Heap
5
6     def match(self, side, qty, limit_price):
7         # Implementation of Price-Time Priority
8         remaining_qty = qty
9         while remaining_qty > 0:
10             # ... Matching logic ...
```

### B Agents (agents.py)

```
1 class MomentumTrader(Agent):
2     def get_action(self, snapshot):
3         # MA Crossover Logic
4         if short_ma > long_ma:
5             return {'side': 'buy', 'type': 'market'}
```

### C RL Environment (rl\_environment.py)

```
1 class TradingEnv(gym.Env):
2     def step(self, action):
3         # Reward Engineering
4         reward = pnl - (0.1 * abs(inventory))
5         return obs, reward, done, {}
```